

LAB_10 - Final LAB

Backtracking, Optimization, DP & Greedy Algorithms

Session Objectives

By the end of this session, you will be able to:

1. Apply backtracking with pruning to solve constrained social network problems
2. Implement dynamic programming for budgeted optimization
3. Compare greedy heuristics vs exact methods in terms of optimality and scalability
4. Choose the right algorithmic strategy based on problem size and constraints
5. Analyze the trade-off between solution quality and computation time

Core Exercises

Exercise 1: Event Invitation – Maximize Attendance with Conflicts (Ref. Maximum Independent Set)

You are organizing an exclusive event. Some users cannot attend together due to personal conflicts (e.g., ex-friends, rivals). You want the largest possible invited set such that no conflicting pair is both invited.

Graph representation:

- Nodes: users
- Edges: conflict (cannot both attend)
- Input: adjacency list or matrix

Requirements:

1. Implement `is_valid_invitation(invited, graph)`
 - Input: list of selected user IDs, graph representation
 - Output: True if no edge exists between any two invited users
 - Complexity target: $O(E)$ or $O(k^2)$ where $k = \text{len}(\text{invited})$
2. Implement `find_max_invitations_exact(graph)`
 - Finds the exact maximum independent set using backtracking with pruning
 - Must be correct for small graphs ($N \leq 40$)
 - Pruning: stop if remaining nodes + current set $\leq$ best found
 - Return: (max_size, invited_list)
3. Implement `find_max_invitations_greedy(graph)`
 - Greedy heuristic: repeatedly pick the node with smallest degree, add to set, remove it and its neighbors
 - Return: (size, invited_list)
 - Compare its result with the exact solution for small graphs

Complexity Analysis Questions:

1. Verification vs construction

- What is the time complexity of verifying a given invited set?
 - Why is verification considered "easy" even if finding the maximum set is hard?
 - Give a concrete small graph example where verification is trivial but finding the optimum requires checking many subsets.
2. Growth of runtime
 - How many subsets exist for $N = 20$, $N = 30$, $N = 40$?
 - Why is $N = 100$ impossible with brute force even with pruning?
 - How does the greedy algorithm scale for $N = 1,000,000$?
 3. Optimality guarantee
 - Can the greedy algorithm always find the maximum independent set? Prove or give a counterexample.
 - For large graphs (millions of nodes), what do real social networks do instead of searching for the exact maximum?

Exercise 2: Viral Message Timing – Maximize Total Reach (Ref. 0/1 Knapsack)

You can send a promotional message to users. Each user i has:

- $\text{cost}[i]$ (server cost in dollars, integer)
- $\text{reach}[i]$ (number of people influenced, integer)

You have a fixed budget. Choose a subset of users to maximize total reach, subject to total cost $\leq$ budget.

Requirements:

1. Implement `maximize_reach_exact(budget, costs, reaches)`
 - Exact solution using dynamic programming (0/1 knapsack)
 - Return: (max_reach, selected_users_list)
 - Complexity: $O(N \times \text{budget})$
 - Works for $\text{budget} \leq 10,000$ and $N \leq 500$
2. Implement `is_within_budget(selection, costs, budget)`
 - Simple helper: sum of costs of selected users $\leq$ budget
 - Complexity: $O(N)$
3. Implement `maximize_reach_greedy(budget, costs, reaches)`
 - Greedy by reach/cost ratio (pick highest ratio first)
 - Return: (reach, selected_users)
 - Compare with exact solution for small instances

Complexity Analysis Questions:

1. Scaling with N
 - For DP: why is runtime $N \times \text{budget}$, not exponential in N ?
 - What happens if $N = 10,000$ and $\text{budget} = 1,000,000$? Is DP still feasible?
 - Why does this problem become harder if costs are real numbers instead of integers?
2. Optimality in practice
 - For $N = 100$, $\text{budget} = 10,000$, DP runs in $\sim 1\text{M}$ operations (fast).
 - For $N = 1,000,000$, $\text{budget} = 1\text{e}9$: is DP possible? Why not?
 - When would you accept the greedy approximation?
3. Exact vs fast comparison
 - Construct a small counterexample where greedy by ratio fails to find the optimum.

- What is the worst-case approximation factor of the ratio-based greedy?
- In real ad platforms, why might they still use heuristics instead of exact DP?

Exercise 3: Group Formation – Minimize External Friends (Ref. Balanced Minimum Cut)

You want to split users into two groups (e.g., for a debate tournament). The goal: minimize the number of friendships between groups (friends separated). However, groups must be balanced: each group must contain at least 40% of all users.

Graph representation:

- Nodes: users
- Edges: friendships (undirected)
- Input: adjacency list or matrix

Requirements:

1. Implement `count_cross_edges(groupA, groupB, graph)`
 - Input: two lists of user IDs, graph representation
 - Output: number of edges where one endpoint is in groupA and the other in groupB
 - Complexity target: $O(E)$
2. Implement `find_balanced_partition_greedy(graph)`
 - Start with a random split respecting the 40% balance constraint
 - Repeatedly move a single user from one group to the other if it reduces cross edges
 - Stop when no improvement is possible
 - Return: (cross_edges, groupA, groupB)
3. Implement `find_balanced_partition_local_search(graph, iterations)`
 - Run the greedy algorithm multiple times with different random initial splits
 - Keep the best solution found
 - Return: (cross_edges, groupA, groupB)

Complexity Analysis Questions:

1. When balance is violated
 - What happens if you try to split a graph of 10 users with a 40% constraint?
 - How can you quickly check if a split respects the balance constraint?
 - Give an example graph where the optimal cut ignores balance, but the balanced version forces a worse cut.
2. Effect of graph density
 - For a complete graph K_n , what is the minimum number of cross edges with balanced groups?
 - For an empty graph (no edges), what is the minimum number of cross edges?
 - Which type of graph is easier to partition? Why?
3. Greedy vs local search
 - Is verifying a given partition's cross edges easier or harder than finding a good partition? Give complexity.
 - Why might the greedy algorithm get stuck in a local minimum?
 - How does increasing the number of iterations affect runtime and solution quality?

Final Integration Questions

1. Classification

For each exercise, classify the core problem as: P – NP – NP-Complete – NP-Hard

2. Decision vs Optimization

Transform each into a decision problem: "Does a solution exist with value $\geq k$ (or $\leq k$)?"

Explain why this matters for complexity theory.

3. Scalability

What happens when: users = 1,000, or users = 1,000,000? Which problems fail first with exact methods? Which can still be solved?

4. Engineering Perspective

- When is approximation acceptable?
- What would real social network platforms do for each of the three problems to scale?

5. Reflection

- Why do some problems resist efficient solutions? What common structure do NP-Complete problems share?
- If $P = NP$ were proven, what would change for these three social network problems?
- What does NP-hardness imply for AI systems trying to "optimize" social network recommendations or group formation?

6. Edge Cases

- Empty graph (no conflicts, no friendships)
- Fully connected graph (everyone conflicts, everyone friends)
- Budget = 0
- Very sparse vs. very dense graphs
- $N = 1$ (single user)

General Instructions

Repository Setup

Create branch: LAB_10_BT_OPT_DP_GA

File Structure:

Follow the same structure of LAB_09

Git Requirements:

Same as for LAB_09

Documentation & Notation

At the End of the Session	Before Friday Midnight of the Same Week
Note: 5	Note: 15
• Each Team (of number BB) Upload at the end of the session a very short PDF document and name it TBB_LAB10_AES.pdf, containing for each exercise: the algorithm in Pseudo-Code (according to the rules explained in lecture 1) – Brief answers of the questions, • You can implement your algorithms to make sure that they are working, • NO AI-generated content (will be checked for authenticity), • NO code snippets.	• You finish all the implementation of all exercises, • You finalize the previous report and give it a new name TBB_LAB10_BFM.pdf and upload it before Friday midnight of the same week, containing for each exercise: same as previous + more complexity analysis and reflection + Test set for edge cases of your Algorithms' implementation, • AI-generated content is not recommended, • A link to the Team's Repository in GitHub.